CHAPTER 20

AI ENGINEERING ROADMAP · STUDY NOTES

Subgraphs in LangGraph

How to embed one graph inside another — the foundation of modular, multi-agent LangGraph systems.

WHAT THIS CHAPTER COVERS

- What a subgraph is, conceptually
- Why subgraphs matter for complex workflows
- Subgraphs in multi-agent architectures
- Modularity, reusability, maintainability
- Failure isolation, state separation, observability
- Two implementation mechanisms
- Isolated-state vs. shared-state subgraphs
- Persistence, streaming & state inspection

Subgraphs in LangGraph

RECAP — CHAPTER 19

Chapter 19 covered Human-in-the-Loop (HITL): pausing a graph mid-execution with `interrupt()` and resuming it with `Command(resume=...)`, backed by a checkpointer. This chapter moves to a different kind of structural tool — **subgraphs**, which let an entire graph be embedded as a single node inside another graph, and which become essential once you start building multi-agent systems.

1. What Is a Subgraph?

Definition

So far, every graph built in this series has used ordinary nodes, where each node represents a single task — calling an LLM, retrieving from a vector database, or calling a tool. A *subgraph* is what you get when, instead of a node representing a single task, that node is **replaced by an entire graph**.

IMPORTANT

A subgraph in LangGraph usually means **a graph that is embedded and executed as a node inside another graph**. The larger graph that contains it is called the **parent graph**. The relationship is purely structural — a subgraph is not a different kind of object, it is an ordinary, independently compiled `StateGraph` that happens to sit inside another one.

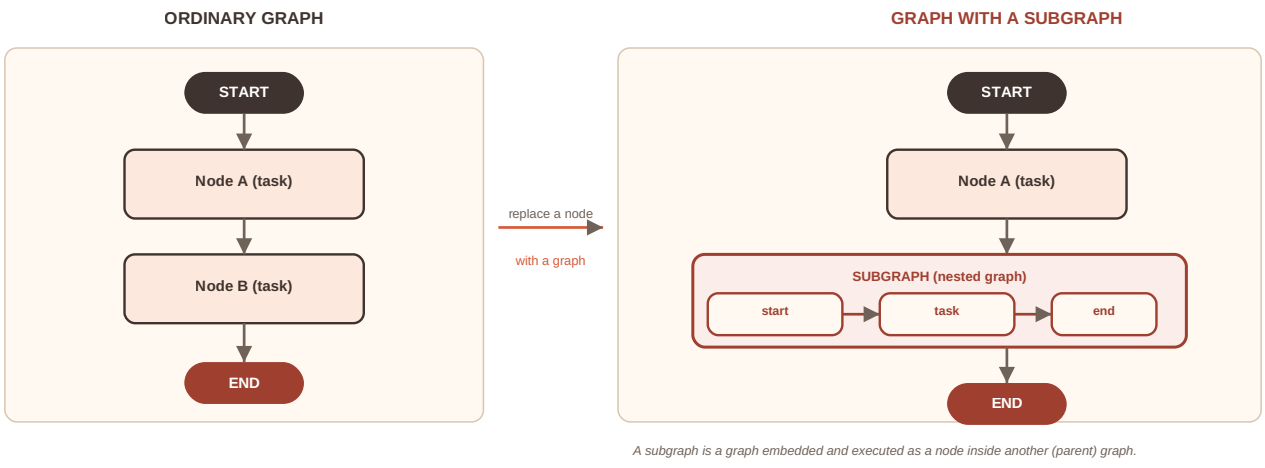


Figure 20.1 — A node that represents a single task can be replaced by an entire nested graph: that nested graph is the subgraph.

↻ Quick Revision — What Is a Subgraph?

- A subgraph is a graph embedded and executed as a node inside another (parent) graph.
- Every node normally represents one task; a subgraph replaces that single-task node with a full graph.
- The containing graph is called the parent graph; the embedded graph is the subgraph.
- A subgraph is still just an ordinary, independently compiled StateGraph.

2. Why Subgraphs Are Needed

The Problem: Growing Complexity

Early GenAI applications typically follow a trivial flow: a user query goes to an LLM, and the LLM returns an output. As more concepts are layered on — tool calling, RAG, conditional routing, retries, memory, Human-in-the-Loop, evaluation, guardrails — a real-world GenAI application can become genuinely large, with a lot happening at once.

A Motivating Example: A Software Development Agent

Consider building an agent for a company whose job is to build software. A real software development process looks roughly like this: a **team lead** assigns work to a **backend team** and a **frontend team**; once both finish, the software is **tested**; after testing, the code is **reviewed**; finally it is handed to **DevOps** to be deployed and monitored.

If this entire process were built as a single LangGraph graph, that graph would become extremely complex — each stage might need its own tools, its own retry logic, its own memory, its own HITL checkpoints, and its own guardrails, all tangled into one graph.

The Solution: Break It Down

The large agent can instead be divided into smaller, focused agents: the team lead becomes a **Planning Agent**; the backend and frontend teams become two separate **Coding Agents**; testing becomes a **Testing Agent**; review becomes a **Code Review Agent**; and deployment becomes a **DevOps Agent**. This produces a *multi-agent architecture*, and each individual agent is naturally represented as a subgraph.



Figure 20.2 — A single complex agent broken down into six focused agents, each implemented as its own subgraph.

↻ Quick Revision — Why Subgraphs Are Needed

- Real-world GenAI applications grow complex: tools, retries, memory, HITL, guardrails, evaluation can all stack up.
- A single giant graph holding all of that logic becomes hard to build and hard to reason about.
- The fix is to decompose a large agent into smaller, focused agents (a multi-agent architecture).
- Each individual agent in a multi-agent system is naturally represented as a subgraph.

3. Core Benefits of Subgraphs

Three Conceptual Benefits

Breaking a large agent into subgraphs, as in the software development example, achieves three things that apply to software design in general:

1. MODULARITY

The entire codebase is broken down into clearly separated functional units. Just as general software engineering advice says code should be modular, the same principle applies to agent graphs — each subgraph owns one well-defined responsibility.

2. REUSABILITY

A subgraph built for one purpose can be reused elsewhere. For example, the same "coding agent" subgraph used for the backend team can be reused for the frontend team — both are essentially doing coding, just against a different set of files.

3. MAINTAINABILITY

Debugging becomes far easier. If a problem occurs, the exact subgraph responsible can be identified and opened directly, rather than searching through one enormous graph.

BEST PRACTICE

When a workflow starts to accumulate multiple distinct responsibilities (its own tools, its own retry logic, its own memory), that is a strong signal it should be pulled out into its own subgraph rather than grown further inside the parent graph.

↻ Quick Revision — Core Benefits

- Modularity: the codebase is broken into clear, separated functional units.
- Reusability: the same subgraph (e.g. a coding agent) can be reused across multiple contexts.
- Maintainability: a problem can be traced directly to the responsible subgraph and debugged there.
- These three benefits are general software-engineering principles applied to agent graphs.

4. LangGraph-Specific Benefits

Beyond the three conceptual benefits, LangGraph provides three additional, framework-level benefits once subgraphs are used.

1. FAILURE ISOLATION

LangGraph is designed so that if one particular subgraph fails or runs into a problem, the rest of the parent graph can still execute — typically with warnings, but without the entire graph collapsing. Without subgraphs, a single node failure can disrupt the rest of the graph.

2. STATE SEPARATION

Every graph in LangGraph has its own state — data about that graph. If the entire software development agent were built as one single graph, there would be only one single state, and every component (coding, testing, DevOps) would interact with that same shared state, which is undesirable. Subgraphs let each agent define its own state, so states do not mismatch across unrelated components.

3. OBSERVABILITY

Tools such as LangSmith can trace a parent graph’s execution end to end. With subgraphs, LangGraph also allows tracing at a granular, per-subgraph level — for example, tracing the coding agent separately from the testing agent or the code review agent, to see token consumption and average latency for each one individually.

REAL WORLD INSIGHT

Per-subgraph tracing is especially valuable in production multi-agent systems: it lets a team answer "which agent is slow or expensive?" instead of only "is the whole pipeline slow or expensive?"

↻ Quick Revision — LangGraph-Specific Benefits

- Failure isolation: a failing subgraph does not necessarily bring down the entire parent graph.
- State separation: each subgraph can define its own state instead of sharing one global state.
- Observability: subgraphs can be traced individually (e.g. in LangSmith), down to token usage and latency per agent.
- Together with modularity, reusability, and maintainability, these form the full case for subgraphs.

5. Two Ways to Implement Subgraphs

LangGraph's documentation describes two different ways a parent graph and a subgraph can communicate. The choice between them is the single most important design decision when adding a subgraph.

MECHANISM 1 — INVOKE A GRAPH FROM A NODE

The subgraph is built completely independently of the parent graph, with its own state schema. Inside one node of the parent graph, the compiled subgraph is simply **invoked** (called) like a function, and its result is used to update the parent state. There is no direct structural connection between the two graphs — only a function call inside a node.

MECHANISM 2 — ADD A GRAPH AS A NODE

The compiled subgraph is passed **directly as a node** in the parent graph, in place of an ordinary node function. In this mode, the subgraph **shares state keys with the parent** — there is no separate schema and no explicit invoke() call.

Aspect	Mechanism 1 — Invoke from Node	Mechanism 2 — Add as Node
State	Parent and subgraph have separate, independent states	Subgraph shares the parent's state keys directly
Connection	Subgraph is called with `.invoke()` inside a node function	Subgraph is passed directly to `add_node()`
Code in the node	A small wrapper function that invokes the subgraph and maps its result back	No wrapper function needed at all

IMPORTANT

The core distinction to remember: in Mechanism 1 the two graphs have their **own separate states**; in Mechanism 2 the subgraph works directly with the **parent's own state keys**. Everything else follows from that one difference.

↻ Quick Revision — Two Implementation Mechanisms

- Mechanism 1 (invoke from a node): subgraph built with its own state, called via .invoke() inside a parent node.
- Mechanism 2 (add as a node): the compiled subgraph is passed directly to add_node(), sharing the parent's state keys.
- Mechanism 1 => isolated states. Mechanism 2 => shared state.
- Both mechanisms are demonstrated using the same use case: generate an English answer, then translate it to Hindi.

6. Mechanism 1 in Practice: Isolated-State Subgraph

Use Case

A user asks a question. An LLM generates an English answer. That English answer must then be translated into Hindi by a second LLM call before being shown to the user.

Building the Subgraph First

The subgraph only needs to know about translation — it takes English text in and produces Hindi text out. Its state is defined independently of the parent.

BEST PRACTICE

Build and compile the subgraph **before** the parent graph. Since the parent will later call into the subgraph, the subgraph needs to exist first — designing it in isolation also forces a clean, minimal state schema (just `input_text` and `translated_text` here) instead of one that accidentally depends on parent-only data.

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_groq import ChatGroq
from dotenv import load_dotenv

load_dotenv()

# subgraph's own, independent state
class SubState(TypedDict):
    input_text: str
    translated_text: str

subgraph_llm = ChatGroq(model='openai/gpt-oss-safeguard-20b')

def translate_text(state: SubState):
    prompt = f"""
        Translate the following text to Hindi.
        Keep it natural and clear. Do not add extra content.

        Text:
        {state['input_text']}
        """.strip()

    response = subgraph_llm.invoke(prompt)
    return {'translated_text': response.content}

subgraph_builder = StateGraph(SubState)
subgraph_builder.add_node('translate_text', translate_text)
subgraph_builder.add_edge(START, 'translate_text')
subgraph_builder.add_edge('translate_text', END)

subgraph = subgraph_builder.compile()
```

The subgraph has exactly one node (`translate_text`), a prompt instructing the model to translate naturally without adding extra content, and is compiled independently — at this point it is a fully working, standalone graph.

Building the Parent Graph

The parent graph defines its own, separate state, with three keys: the original question, the English answer, and the Hindi answer.

```
class ParentState(TypedDict):
    question: str
    answer_eng: str
    answer_hin: str

parent_llm = ChatGroq(model='openai/gpt-oss-safeguard-20b')

def generate_ans(state: ParentState):
    answer = parent_llm.invoke(
        f"You are a helpful assistant. Answer clearly.\n\nQuestion: {state['question']}"
    ).content
    return {'answer_eng': answer}

def translate_ans(state: ParentState):
    # this is the entire connection to the subgraph
    result = subgraph.invoke({'input_text': state['answer_eng']})
    return {'answer_hin': result['translated_text']}

parent_builder = StateGraph(ParentState)
parent_builder.add_node('generate_ans', generate_ans)
parent_builder.add_node('translate_ans', translate_ans)

parent_builder.add_edge(START, 'generate_ans')
parent_builder.add_edge('generate_ans', 'translate_ans')
parent_builder.add_edge('translate_ans', END)

graph = parent_builder.compile()
graph.invoke({'question': 'What is quantum physics'})
```

Inside `translate_ans`, nothing is done directly — the function simply calls `subgraph.invoke(...)`, passing the English answer as the subgraph's `input_text`. The subgraph returns its own complete final state, from which only `translated_text` is extracted and written into the parent's `answer_hin` key.

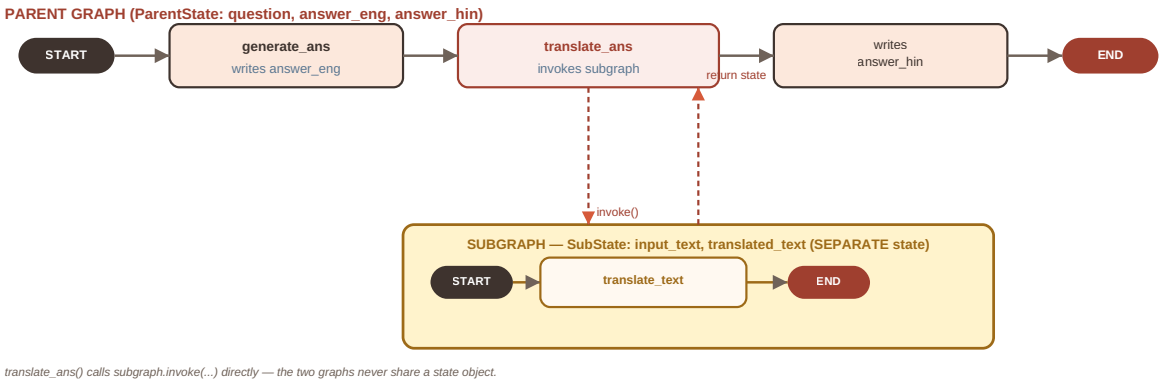


Figure 20.3 — Mechanism 1: the translate_ans node invokes the independently compiled subgraph, which has its own, separate SubState.

> ↻ **Quick Revision — Mechanism 1 (Isolated State)**
>
> • Subgraph is built and compiled completely independently, with its own TypedDict state (SubState).
>
> • Parent node (translate_ans) calls subgraph.invoke({...}) and manually extracts the field it needs.
>
> • ParentState and SubState never merge; only the data explicitly passed in and extracted back crosses the boundary.
>
> • Result: graph.invoke() returns question, answer_eng, and answer_hin, all inside ParentState.

7. Mechanism 2 in Practice: Shared-State Subgraph

Same Use Case, Different Mechanism

The same translation use case is now rebuilt using Mechanism 2. The key difference: there is only **one** state definition — `ParentState` — and the subgraph is built directly against it instead of a separate schema.

```
class ParentState(TypedDict):
    question: str
    answer_eng: str
    answer_hin: str

parent_llm = ChatGroq(model='openai/gpt-oss-safeguard-20b')
subgraph_llm = ChatGroq(model='openai/gpt-oss-safeguard-20b')

# subgraph node reads/writes ParentState keys directly — no SubState
def translate_text(state: ParentState):
    prompt = f"""Translate the following text to Hindi.
            Keep it natural and clear. Do not add extra content.

            Text:
            {state['answer_eng']}""".strip()

    translated_hin = subgraph_llm.invoke(prompt).content
    return {'answer_hin': translated_hin}

subgraph_builder = StateGraph(ParentState)
subgraph_builder.add_node('translate_text', translate_text)
subgraph_builder.add_edge(START, 'translate_text')
subgraph_builder.add_edge('translate_text', END)

subgraph = subgraph_builder.compile()
```

Notice that `translate_text` is typed against `ParentState`, not a separate `SubState` — it reads `answer_eng` straight from the parent state and writes `answer_hin` straight back into it.

Wiring the Subgraph Into the Parent as a Node

```
def generate_answer(state: ParentState):
    answer = parent_llm.invoke(
        f"You are a helpful assistant. Answer clearly.\n\n Question: {state['question']}"
    ).content
    return {'answer_eng': answer}

parent_builder = StateGraph(ParentState)

parent_builder.add_node("answer", generate_answer)
parent_builder.add_node("translate", subgraph)   # ← the subgraph IS the node

parent_builder.add_edge(START, 'answer')
parent_builder.add_edge('answer', 'translate')
parent_builder.add_edge('translate', END)

graph = parent_builder.compile()
graph.invoke({'question': 'What is quantum mechanics?'})
```

The biggest change is on the line `add_node("translate", subgraph)`: instead of passing an ordinary Python function, the compiled `subgraph` object itself is passed as the node. No wrapper function, no explicit `.invoke()` call, and no manual field extraction are needed — LangGraph treats the compiled subgraph as the node's implementation and lets it read and write the shared state directly.

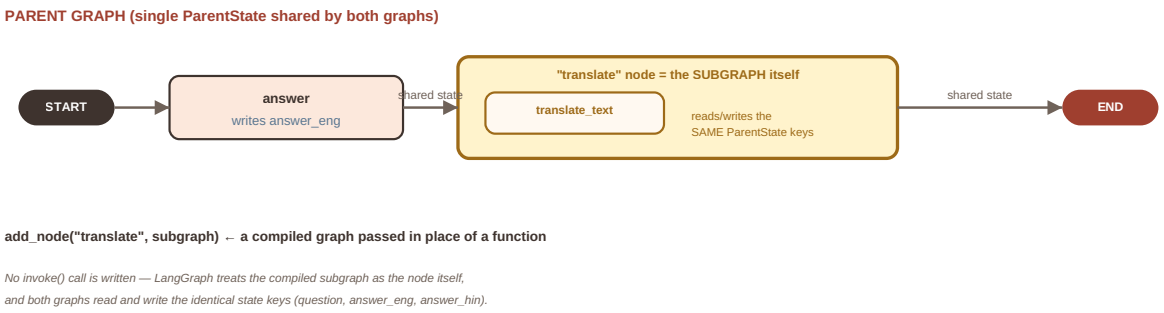


Figure 20.4 — Mechanism 2: the compiled subgraph is passed straight into add_node(), sharing the ParentState keys of the parent graph, with no separate schema.

> **COMMON MISTAKE**
>
> Passing a subgraph straight into `add_node()` (Mechanism 2) only works cleanly when the subgraph's state keys line up with the parent's. If the subgraph needs different or additional keys, Mechanism 1 (an explicit invoke-and-map wrapper) is the safer choice.

↻ Quick Revision — Mechanism 2 (Shared State)

- Only one state schema exists: ParentState. The subgraph's node function is typed against it directly.
- The subgraph is registered with add_node("translate", subgraph) — a compiled graph passed in place of a function.
- No .invoke() call and no manual field extraction are written in the parent graph.
- Both graphs read and write the identical state keys (question, answer_eng, answer_hin).

8. Best Practices: Persistence, Streaming & State Viewing

LangGraph's official documentation covers a few additional subgraph capabilities beyond what a first pass needs — worth knowing about, and worth studying directly in the docs.

ADDING PERSISTENCE TO SUBGRAPHS

A checkpointer only needs to be given to the **parent** graph at compile time. LangGraph automatically checkpoints the subgraph (the "child") as well, with no extra configuration required on the subgraph itself.

STREAMING SUBGRAPH OUTPUT

Just like any other graph, a subgraph's output can be streamed as it is produced, instead of waiting for the entire parent graph to finish.

VIEWING SUBGRAPH STATE

The state of an individual subgraph can be inspected on its own, separately from the parent graph's overall state — useful for debugging a specific agent in a multi-agent system.

> **INTERVIEW TIP**
>
> If asked how checkpointing works with subgraphs, the key fact to state is: **you only attach a checkpointer to the parent graph** — LangGraph propagates checkpointing down to subgraphs automatically.

↻ Quick Revision — Persistence, Streaming & State Viewing

- Give the checkpointer to the parent graph only; LangGraph checkpoints the subgraph automatically.
- Subgraph output can be streamed the same way a normal graph's output is streamed.
- A subgraph's own state can be viewed separately from the parent graph's state.
- These capabilities are documented in LangGraph's official docs under Subgraphs.

K. Key Takeaways

- **Subgraph** — a graph embedded and executed as a node inside another (parent) graph; the parent/subgraph relationship is purely structural.
- **Need for subgraphs** — complex, real-world GenAI applications (e.g. a software development agent) become unwieldy as one giant graph and benefit from being decomposed.
- **Multi-agent architectures** — decomposing one large agent into focused agents (planning, coding, testing, review, DevOps), each naturally represented as a subgraph.
- **Conceptual benefits** — modularity, reusability, and maintainability.
- **LangGraph-specific benefits** — failure isolation, state separation, and granular observability (per-subgraph tracing).
- **Mechanism 1 — Invoke from a node** — subgraph built with its own independent state, called via `.invoke()` inside a parent node function.
- **Mechanism 2 — Add as a node** — a compiled subgraph passed directly to `add_node()`, sharing the parent's state keys with no separate schema.
- **Persistence** — a checkpointer attached to the parent graph is automatically propagated to its subgraphs.

R. Revision Sheet

Subgraph

A graph embedded and executed as a node inside another (parent) graph.

Reusability

The same subgraph (e.g. a coding agent) can be reused across multiple contexts.

Parent Graph

The outer graph that contains one or more subgraphs as nodes.

Maintainability

Problems can be traced directly to the responsible subgraph and fixed there.

Modularity

Large workflows broken into clearly separated, focused functional units.

Failure Isolation

A failing subgraph does not necessarily bring down the rest of the parent graph.

State Separation

Each subgraph can define its own state instead of sharing one global state.

Mechanism 2 (Shared State)

Compiled subgraph passed directly to `add_node()`; shares the parent's state keys.

Observability

Subgraphs can be traced individually — tokens and latency per agent.

Checkpointing Subgraphs

Attach the checkpointer only to the parent graph; the subgraph is checkpointed automatically.

Mechanism 1 (Isolated State)

Subgraph has its own state; invoked via `.invoke()` inside a parent node.

Use Case in This Chapter

Generate an English answer, then translate it to Hindi via a dedicated translation subgraph.

I. Interview Questions

Q1. What is a subgraph in LangGraph, and how does it relate to a parent graph?

Q2. Why do complex, real-world GenAI applications tend to need subgraphs rather than one single large graph?

Q3. How would you decompose a complex agent (e.g. a software development agent) into a multi-agent architecture using subgraphs?

Q4. What are the three core software-design benefits subgraphs provide, and what does each one mean?

Q5. What are the three LangGraph-specific benefits of using subgraphs, beyond the general software-design benefits?

Q6. What are the two mechanisms for implementing a subgraph in LangGraph, and what is the core difference between them?

Q7. In the "invoke from a node" mechanism, how does data pass between the parent state and the subgraph's state?

Q8. In the "add as a node" mechanism, why is no explicit `.invoke()` call needed inside the parent graph?

Q9. How does checkpointing/persistence work when a parent graph contains a subgraph?

Q10. What observability benefit does LangGraph provide for subgraphs, and why is it useful in a multi-agent system?

F. Further Reading

- LangGraph official documentation — "Subgraphs" (how-to guide covering both communication mechanisms, persistence, and streaming).
- LangGraph official documentation — "Multi-agent systems" (architectural patterns that build directly on the subgraph concept).
- LangSmith documentation — tracing nested/subgraph executions for per-agent observability.